

RAMMA — Project Status Report (Review 1 + Review 2)

Requirement-Aware ML Monitoring & Automation

This document summarizes what has been built, verified, and evaluated so far, and what remains before Review 3. Written for continuity across chat sessions / new contributors picking up this repo.

1. What RAMMA Does

RAMMA converts a plain-English ML monitoring requirement (e.g., "*Recall must remain above 93%*") into a structured, calibrated, executable monitoring policy, and generates plain-English alerts with suspected drift contributors when violations occur — without replacing existing monitoring infrastructure (Evidently/MLflow-style tools).

2. Architecture (Implemented Components)

Component	File	Status
Metric Library (single source of truth for supported metrics)	<code>src/ramma_nlp/metric_library.py</code>	Done
Pydantic Schema + Validators	<code>src/ramma_nlp/schema.py</code>	Done — includes a validator preventing silent <code>[0.0]</code> threshold defaults
Requirement Interpreter (LLM-based, Gemini)	<code>src/ramma_nlp/interpreter.py</code>	Done — dual-path: Gemini structured output + rule-based fallback
Ambiguity Detector	(part of <code>interpreter.py</code>)	Done
Backend (FastAPI + SQLAlchemy/SQLite, Postgres via Docker)	<code>src/ramma_backend/</code>	Done

Component	File	Status
Threshold Engine (percentile calibration: mean – 2×std)	<code>src/ramma_backend/threshold_engine.py</code>	Done
Monitor Generator (dual-threshold config: business requirement + operational baseline)	<code>src/ramma_backend/monitor_generator.py</code>	Done
Monitoring Runtime	<code>src/ramma_backend/monitoring_runtime.py</code>	Done
Alert & Explanation Engine (deterministic violation check + LLM phrasing)	<code>src/ramma_nlp/explain.py</code>	Done
Suspected Contributor Ranking (PSI-based)	<code>src/ramma_nlp/contributors.py</code>	Done
End-to-End CLI Demo	<code>cli.py</code>	Done — <code>python cli.py demo</code>
Synthetic dataset + trained classifier + injected drift	<code>data/prepare_dataset.py</code>	Done

Key design decision: the pipeline treats the ML model as a black box. Only the Monitoring Runtime ever calls the model (read-only, via `.predict()`/`.predict_proba()`). Every other component works on text, statistics, or config — never on model internals. This keeps RAMMA reusable across any sklearn-compatible classifier.

Key design decision #2 (dual-threshold system): RAMMA tracks two thresholds separately rather than one:

- `business_requirement_threshold` — the original stated target (e.g., 93%)
- `operational_baseline_threshold` — calibrated from reference data (e.g., 92.48%)
- `requirement_met_by_baseline` — whether the model currently satisfies the business ask at all

This was added after discovering that naive threshold calibration can silently substitute a weaker threshold when a model doesn't meet its stated business requirement — RAMMA now reports both signals honestly instead of hiding requirement non-compliance behind a "no violation" alert.

Requirement Interpreter Accuracy

Dataset	Size	Accuracy	Ambiguity Precision	Ambiguity Recall
Original labeled set	30	90.00% (post-fix; was 86.67% pre-fix)	100% (post-fix; was 86.67%)	100%
Hand-written cold stress-test set	10	100.00% (post-fix; was 50.00% pre-fix)	100% (post-fix; was 44.44%)	100%

Important caveat, stated honestly: the stress-test set's 100% score was measured on the *same* 10 sentences used to diagnose and fix the two bugs below — it confirms the specific fixes worked, but has not yet been validated against a genuinely fresh, independently-phrased set. Treat the 90% original-set score as the stronger generalization signal, since those rows were not used to tune the fix.

Bugs Found and Fixed During Stress-Testing

1. **Regex threshold extraction only matched the `%` symbol**, failing on spelled-out "percent"/"per cent"/"pct" — caused valid, precise requirements to be misclassified as ambiguous. Fixed with an expanded regex pattern.
2. **Substring keyword collision:** the ambiguity check for the word "low" matched inside "below" (as in "must stay below 2%"), incorrectly flagging clear requirements as vague. Fixed with word-boundary regex matching.
3. **Silent `0.0` threshold defaulting:** when extraction failed on a non-ambiguous requirement, the system silently returned `threshold=0.0` instead of failing loudly — a dangerous failure mode since `0.0` could be mistaken for a real, intentional value downstream. Fixed by adding a Pydantic validator that raises an error instead.

Model / Drift Detection Verification

Dataset	Recall Precision	Business Requirement (93%)	Operational Baseline (92.48%)
<code>production_clean.csv</code>	94.15% 83.91%	✅ Met	✅ Met
<code>production_drifted.csv</code>	74.03% 48.10%	❌ Violated	❌ Violated

- Drift injection (feature shifts + label flips on `location_distance_km`, `transaction_amount`, `device_risk_score`) produces a confirmed 20-point recall drop.

- Suspected Contributor ranking correctly identifies `location_distance_km` (PSI = 0.53) as the top contributor, matching the actual injected perturbation.
- **Honest limitation:** the classifier's hyperparameters (`class_sep=2.3`, class weight `{0:1, 1:10}`, decision cutoff `0.35`) were deliberately tuned via grid search so that clean-data recall lands just above the 93% business requirement. This makes the demo narrative clear but does not reflect real-world fraud-detection difficulty — say this proactively if asked.

Test Suite

- 43/43 automated unit + integration tests passing.
- Full CLI demo (`python cli.py demo`) runs end-to-end with exit code 0.

4. What's NOT Yet Built (Deliberately Deferred)

- Full traceability graph (currently a simple relational link, not a graph DB)
- Change-impact analysis across requirement/model updates
- Support beyond binary classification (regression, multi-class)
- Streamlit/UI demo layer (currently CLI-only — planned for Review 3)
- Literature survey write-up (content already researched: RAMP (CHI 2026) as closest related work, NL-to-formal-spec papers as background — needs to be written into slides/report, not code)

5. What Would Improve Results Before Review 2 / Into Review 3

Ranked by impact vs. effort:

1. **Validate stress-test generalization on a genuinely fresh set.** Write 3-5 NEW hand-written requirements (not derived from the same patterns as the existing 10) and re-run the evaluator. This closes the "same set used to diagnose and fix" gap noted above and gives a stronger, more defensible generalization claim.
2. **Expand the labeled dataset beyond 30 examples.** Both accuracy numbers are based on small N — more real-world-style examples (aim for 60-100) would tighten confidence in the reported accuracy and ambiguity precision/recall.
3. **Verify the evaluation script's scoring logic for ambiguous rows.** Confirm `evaluate_interpreter.py` only checks `is_ambiguous` for rows where the expected value is `True`, and doesn't also silently score the (irrelevant) guessed metric/threshold on those rows — worth a direct code read to confirm the 100% ambiguity precision isn't partially an artifact of lenient scoring.

4. **Address the tuned/synthetic classifier honestly in the presentation**, or alternatively retrain against a more realistic (less hand-tuned) dataset if time allows, so the demo's "drift causes failure" story isn't reliant on hyperparameters chosen specifically to hit target numbers.
5. **Build the Streamlit demo UI** (Review 3 scope) — currently all verification has been CLI-only; a visual, single-screen demo (requirement in → structured rule → live violation → explanation → contributors) will present far better to judges than console output.
6. **Write the literature survey section** citing RAMP (CHI 2026) and NL-to-formal-spec papers, with a one-sentence differentiation ready for Q&A: RAMMA is quantitative ML performance/drift monitoring with data-driven threshold calibration; RAMP is governance/compliance rule generation — related but distinct problems.
7. **Reconcile any remaining duplicate/legacy code from earlier team iterations** before final submission — an earlier version of this project had a second, independently-built interpreter/backend from a teammate's separate track; confirm only the consolidated version in this repo is what's being submitted, and that no dead/duplicate modules remain.
8. **Consider a second demo scenario dataset** (different domain than fraud) if time allows, to show the pipeline isn't narrowly tuned to one use case.

6. How to Run This Project

```
❏  
bash  
# Local dev (SQLite)  
pip install -r requirements.txt  
python data/prepare_dataset.py # generates reference/production datasets + trains classifier  
python cli.py demo             # full end-to-end demo  
python -m pytest               # run test suite  
python scripts/evaluate_interpreter.py data/labeled_requirements.csv  
python scripts/evaluate_interpreter.py data/stress_test_requirements.csv  
  
# Full stack (Postgres via Docker)  
docker compose up
```

Requires a `GEMINI_API_KEY` in a local `.env` file (see `.env.example`) — never commit this file.